

MODULE 4

INTERFACES AND PACKAGES

Contents

- **Interfaces:** Fundamentals, multiple and nested inheritance.
- Java Language:
 - Creating and Implementing an Interface,
 - Using Interface References,
 - Implementing Multiple Interfaces,
 - Extending Interfaces, Nested Interface.
- **Packages:**
 - Package Fundamentals,
 - Java Language: Packages in Java,
 - Member Access, Importing Packages, Static import. File I/O

Interfaces



- Abstraction in Java could be achieved with the following ways.

Interfaces

Abstract Methods and Class

- using interface, you can specify what a class must do, but not how it does it.
- They are similar to classes, but they lack instance variables & their methods are declared **without any body**. **Interface variables are static**
- Interface methods do not have a body - the body is provided by the "implement" class
- Java **does not permit multiple inheritance from classes**, but permits implementation of multiple interfaces.

Why use Java Interfaces



Syntax:

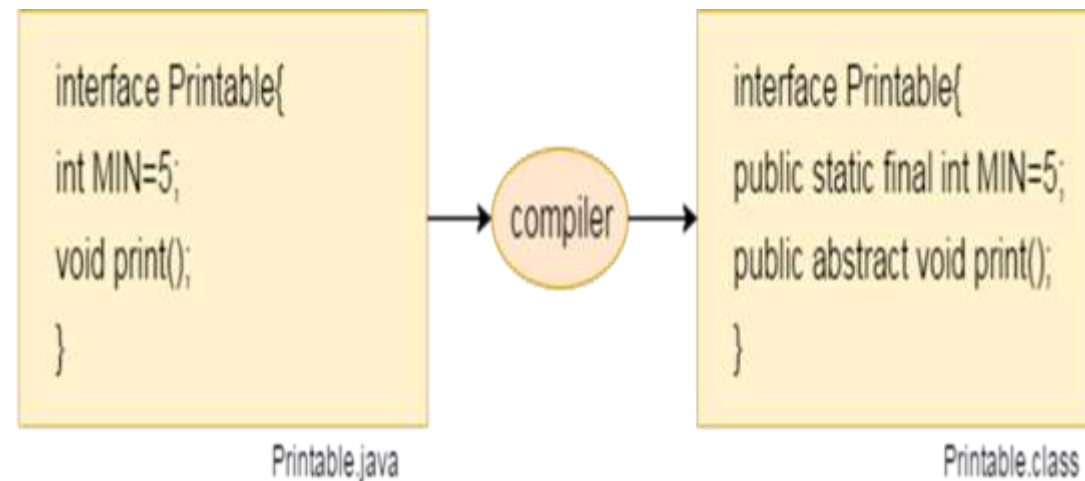
```
interface <interface_name>{
```

```
    methods // declare methods that abstract  
            //public
```

```
    fields  // declare constant fields  
            // by default. Public static final
```

```
}
```

Interface fields are public, static and final by default, and the methods are public and abstract.



Why use Java Interfaces



It is used to achieve abstraction.

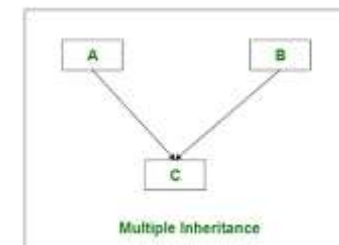
1

2

By interface, we can support the functionality of multiple inheritance.

It can be used to achieve loose coupling.

3



Java Interfaces

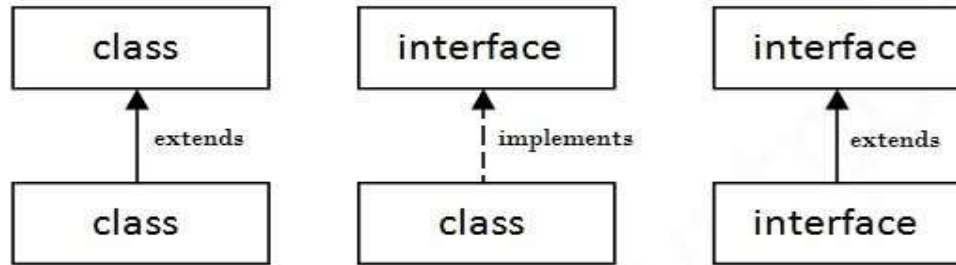


Prior to java 8, [interface in java](#) can only have abstract methods. All the methods of interfaces are public & abstract by default.

Java 8 allows the interfaces to have [default and static methods.](#)

In Java 9, we can create [private methods](#) inside an interface. Interface allows us to declare private methods that help to **share** common code between **non-abstract** methods.

Why use Java Interfaces



Java Interface Example

```
interface printable{  
    void print();  
}  
  
class A6 implements printable{  
    public void print(){System.out.println("Hello");}  
  
    public static void main(String args[]){  
        A6 obj = new A6();  
        obj.print();  
    }  
}
```

Interfaces

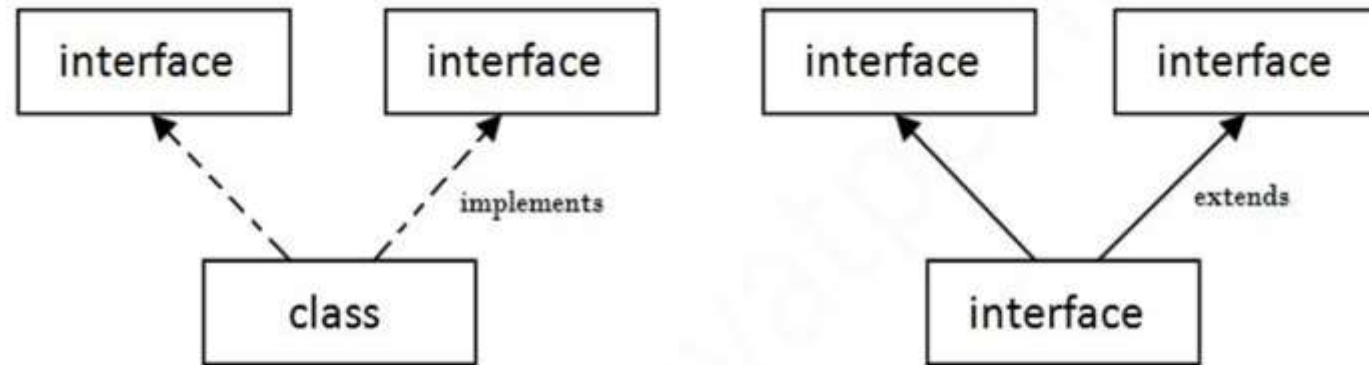


```
interface Bank{  
    float rateOfInterest();  
}  
  
class SBI implements Bank{  
    public float rateOfInterest(){return 9.15f;}  
}  
  
class PNB implements Bank{  
    public float rateOfInterest(){return 9.7f;}  
}  
  
class TestInterface2{  
    public static void main(String[] args){  
        Bank b=new SBI();  
        System.out.println("ROI: "+b.rateOfInterest());  
    }  
}
```


Multiple inheritance in Java by interface



If a class implements multiple interfaces, or an interface extends multiple interfaces, it is known as multiple inheritance.



Multiple Inheritance in Java

Multiple inheritance in Java by interface



```
interface Printable{
    void print();
}

interface Showable{
    void show();
}

class A7 implements Printable,Showable{
    public void print(){System.out.println("Hello");}
    public void show(){System.out.println("Welcome");}

    public static void main(String args[]){
        A7 obj = new A7();
        obj.print();
        obj.show();
    }
}
```

Multiple inheritance in Java by interface



Multiple inheritance is not supported through class in java, but it is possible by an interface

```
interface Printable{  
    void print();  
}  
  
interface Showable{  
    void print();  
}  
  
class TestInterface3 implements Printable, Showable{  
    public void print(){System.out.println("Hello");}  
    public static void main(String args[]){  
        TestInterface3 obj = new TestInterface3();  
        obj.print();  
    }  
}
```

Multiple inheritance in Java by interface



Java 8 Default Method in Interface

```
interface Drawable{  
    void draw();  
    default void msg(){System.out.println("default method");}  
}  
  
class Rectangle implements Drawable{  
    public void draw(){System.out.println("drawing rectangle");}  
}  
  
class TestInterfaceDefault{  
    public static void main(String args[]){  
        Drawable d=new Rectangle();  
        d.draw();  
        d.msg();  
    }  
}
```

Output:

```
drawing rectangle  
default method
```

Nested Interface JAVA(inside interface)



- ✓ An interface can be declared a member of a class or another interface. Such an interface is called a member interface or a nested interface.
- ✓ A nested interface can be declared as public, private, or protected.
- ✓ When a nested interface is used outside of its enclosing scope, it must be qualified by the name of the class or interface of which it is a member.
- ✓ Thus, outside of the class or interface in which a nested interface is declared, its name must be fully qualified.

Nested Interface JAVA(inside interface)



```
interface Showable{
    void show();
    interface Message{
        void msg();
    }
}

public class TestNestedInterface1 implements Showable.Message
{
    public void msg()
    {
        System.out.println("Hello nested interface");
    }

    public static void main(String[] args) {

        // TODO Auto-generated method stub
        Showable.Message message=new TestNestedInterface1();//upcasting here
        message.msg();}
}
```

Packages



A java package is a group of similar types of classes, interfaces and sub-packages.

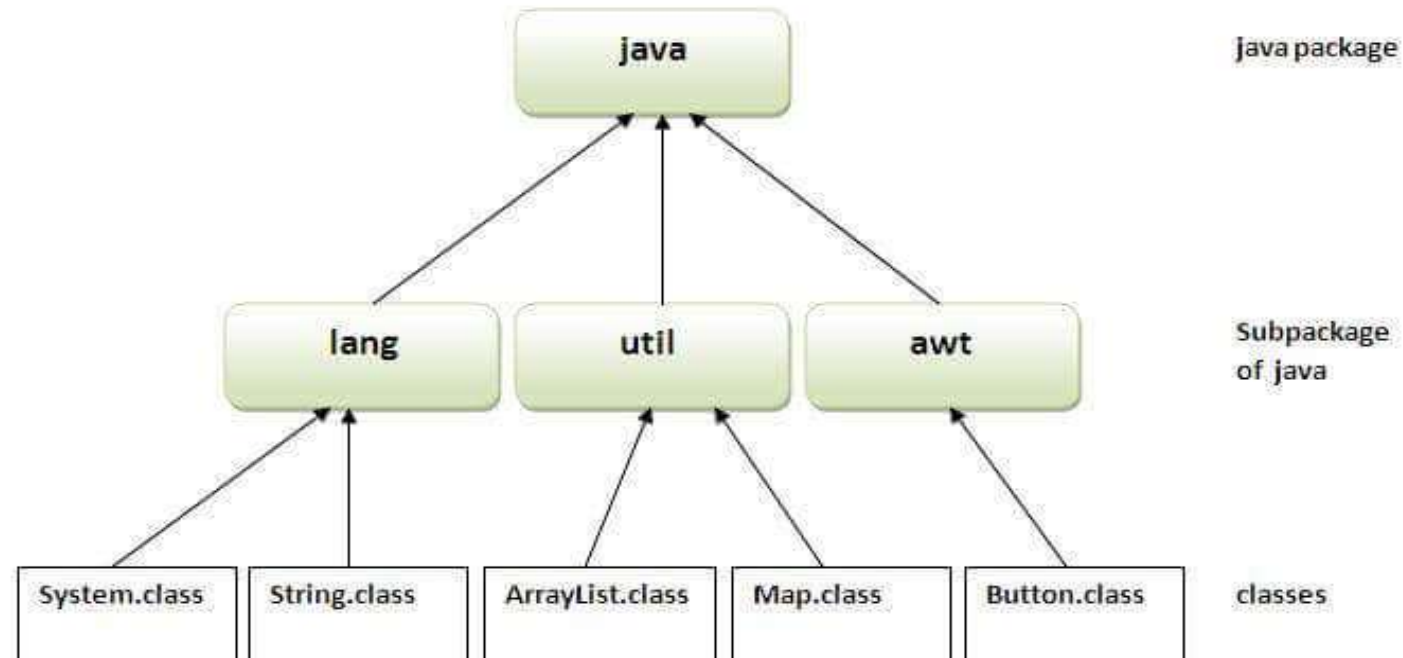
Package in java can be categorized in two form, **built-in package** and **user-defined package**.

There are many **built-in packages** such as java, lang, awt, javax, swing, net, io, util, sql etc.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.

Packages



Packages



The package keyword is used to create a package in java.

```
package mypack; //save as Simple.java
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to package");
    }
}
```

How to compile java package

```
javac -d directory javafilename
```

For **example**

```
javac -d . Simple.java
```

The -d switch specifies the destination where to put the generated class file. You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc. If you want to keep the package within the same directory, you can use . (dot).

Packages



How to access package from another package?

There are three ways to access the package from outside the package.

1. `import package.*;`
2. `import package.classname;`
3. fully qualified name.

Packages



1) Using packagename.*

//save by A.java

```
package pack;  
public class A{  
    public void msg(){System.out.println("Hello");}  
}
```

//save by B.java

```
import pack.*;  
class B{  
    public static void main(String args[]){  
        A obj = new A();  
        obj.msg();  
    }  
}
```

Packages



2)

Using

//save by A.java
package mypack;

package pack;

public class A{

public void msg(){System.out.println("Hello");}

}

//save by B.java

package mypack;

import pack.A;

class B{

public static void main(String args[]){

A obj = **new** A();

obj.msg();

}

}

Packages



3) Using fully qualified name

//save by A.java

```
package pack;
```

```
public class A{
```

```
    public void msg(){System.out.println("Hello");}
```

```
}
```

//save by B.java

```
package mypack;
```

```
class B{
```

```
    public static void main(String args[]){
```

```
        pack.A obj = new pack.A();//using fully qualified name
```

```
        obj.msg();
```

```
    }
```

```
}
```

Packages



static import concept is introduced in 1.5 version. With the help of static import, we can access the static members of a class directly without class name or any object. For Example: we always use `sqrt()` method of `Math` class by using `Math` class i.e. **`Math.sqrt()`**, but by using static import we can access `sqrt()` method directly.

```
import static java.lang.Math.*;
class Test2 {
    public static void main(String[] args)
    {
        System.out.println(sqrt(4));
        System.out.println(pow(2, 2));
        System.out.println(abs(6.3));
    }
}
```

File I/O in Java



- **File I/O (Input/Output)** in Java refers to reading data from files and writing data to files. Java provides several classes in the `java.io` packages to perform these operations.

◆ Basic File I/O Classes (from `java.io` package)

Class	Description
<code>File</code>	Represents a file or directory path.
<code>FileReader</code>	Used to read data from a text file (character by character).
<code>FileWriter</code>	Used to write data to a text file.
<code>BufferedReader</code>	Reads text from an input stream efficiently (line by line).
<code>BufferedWriter</code>	Writes text to an output stream efficiently.
<code>FileInputStream</code>	Reads data from a file in bytes (useful for binary files like images).
<code>FileOutputStream</code>	Writes data to a file in bytes .

File Operations:

Operation	Class/Method
Create a file	<code>File.createNewFile()</code>
Write to a file	<code>FileWriter</code> , <code>BufferedWriter</code>
Read from a file	<code>FileReader</code> , <code>BufferedReader</code>
Append to a file	<code>FileWriter(file, true)</code>
Delete a file	<code>File.delete()</code>

Example : Read from a File (Using FileReader)



```
import java.io.FileReader;
import java.io.IOException;

public class ReadFileExample {
    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("example.txt");
            int data;
            while ((data = reader.read()) != -1) {
                System.out.print((char) data);
            }
            reader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Example : Reading a File Line by Line (Using BufferedReader)



```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class BufferedReadExample {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("example.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Example: Copy Content from One File to Another



```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class CopyFileExample {
    public static void main(String[] args) {
        // Input and Output file names
        String sourceFile = "source.txt";
        String destinationFile = "destination.txt";

        try {
            FileReader reader = new FileReader(sourceFile);
            FileWriter writer = new FileWriter(destinationFile);

            int character;
            // Read from source and write to destination
            while ((character = reader.read()) != -1) {
                writer.write(character);
            }

            System.out.println("File copied successfully!");

            // Close the files
            reader.close();
            writer.close();
        } catch (IOException e) {
            System.out.println("An error occurred while copying the file.");
            e.printStackTrace();
        }
    }
}
```